

RELATÓRIO TÉCNICO

1. Descrição do Problema

O problema do labirinto consiste em determinar um caminho válido e ótimo entre dois pontos dentro de um ambiente representado por uma matriz bidimensional. Nesse contexto, o labirinto é modelado como uma grade composta por linhas e colunas, na qual cada posição representa uma célula do ambiente. Cada célula pode assumir dois valores: 1, indicando que o espaço é livre e pode ser percorrido, ou 0, indicando que se trata de um obstáculo, impossibilitando a passagem. O objetivo é encontrar o menor caminho possível entre uma célula de origem e uma célula de destino, respeitando as restrições impostas pelos obstáculos presentes na matriz.

Do ponto de vista da Inteligência Artificial, esse problema pode ser formalizado como um problema de busca em espaço de estados. Cada estado corresponde à posição atual do agente no labirinto, representada por um par ordenado (linha, coluna). O estado inicial é a posição de origem fornecida pelo usuário, enquanto o estado objetivo é a posição de destino. As ações possíveis consistem nos movimentos para células vizinhas, que podem incluir deslocamentos horizontais, verticais e também diagonais. Cada movimento gera um novo estado, desde que a célula correspondente seja válida, esteja dentro dos limites da matriz e não seja um obstáculo.

Além da definição dos estados e operadores, o problema envolve uma função de custo associada aos movimentos realizados. Em uma abordagem simples, cada deslocamento pode possuir custo unitário, representando um passo no labirinto. Assim, o custo total de um caminho corresponde ao número de movimentos necessários para sair da origem e alcançar o destino. O desafio central consiste em explorar o espaço de estados de maneira eficiente, evitando percorrer caminhos desnecessários e garantindo que a solução encontrada seja a de menor custo possível.

Esse problema se torna computacionalmente desafiador à medida que o tamanho do labirinto aumenta ou quando há muitos obstáculos distribuídos pelo ambiente. Uma busca não informada pode explorar uma grande quantidade de células antes de encontrar a solução, o que compromete a eficiência. Por isso, o uso de algoritmos de busca informada, como o A*, é especialmente adequado. O algoritmo A* combina o custo real acumulado desde a origem até o estado atual, representado por $g(n)$, com uma estimativa heurística do custo restante até o destino, representada por $h(n)$. A soma dessas duas componentes gera a função de avaliação $f(n) = g(n) + h(n)$, que orienta a escolha do próximo estado a ser expandido.

No caso do labirinto, uma heurística comum é a distância euclidiana entre a célula atual e a célula de destino, pois fornece uma estimativa da proximidade em linha reta até o objetivo. Quando essa heurística é admissível e consistente, o algoritmo A* garante

encontrar um caminho ótimo, caso ele exista. Se não houver caminho possível devido à disposição dos obstáculos, o algoritmo percorre os estados acessíveis e, ao esgotar as possibilidades, conclui corretamente que não existe solução.

Portanto, o problema do labirinto é uma abstração clássica de problemas reais de navegação e planejamento de rotas, como a movimentação de robôs em ambientes estruturados, sistemas de GPS e inteligência artificial em jogos digitais. A aplicação do algoritmo A* nesse contexto demonstra como técnicas de busca heurística podem resolver de forma eficiente problemas de caminho mínimo em ambientes com restrições.

2. Como o A* se encaixa com a solução do problema

O problema do labirinto consiste em encontrar um caminho válido entre uma posição inicial e uma posição de destino dentro de uma matriz que representa o ambiente. Nessa matriz, algumas células são livres para movimentação e outras representam obstáculos que não podem ser atravessados. O objetivo é determinar um caminho que leve da origem ao destino respeitando essas restrições, preferencialmente com o menor custo possível, que normalmente corresponde ao menor número de passos.

O algoritmo A* se encaixa de forma adequada nesse problema porque é um método de busca informada, ou seja, utiliza conhecimento adicional para orientar a exploração. Ele avalia cada posição do labirinto com base em uma função de custo composta por dois fatores: o custo já percorrido desde o ponto inicial até o estado atual e uma estimativa do custo restante até o objetivo, calculada por meio de uma heurística. A soma desses dois valores permite ao algoritmo priorizar os caminhos que parecem mais promissores.

Diferentemente dos algoritmos de busca cega, como a busca em largura (BFS) ou a busca em profundidade (DFS), o A* não explora os estados de maneira indiscriminada. A busca em largura, por exemplo, expande todos os nós de um mesmo nível antes de avançar, podendo visitar muitas posições irrelevantes. Já a busca em profundidade pode seguir caminhos longos e inadequados antes de retroceder. O A*, ao utilizar uma heurística que estima a proximidade do objetivo, reduz significativamente o número de estados explorados, concentrando-se nas regiões do labirinto que realmente conduzem ao destino.

Assim, o A* tende a ser mais eficiente em termos de tempo e processamento, pois evita expansões desnecessárias e, quando a heurística é admissível (ou seja, não superestima o custo real até o objetivo), ainda garante que o caminho encontrado seja ótimo. Isso o torna especialmente adequado para problemas de navegação em labirintos e outros cenários de busca em grafos.

3. Correspondência com o pseudocódigo

function BEST-FIRST-SEARCH(*problem*, *f*) returns a solution node or failure

node ← NODE(STATE=*problem*.INITIAL)

frontier ← a priority queue ordered by *f*, with *node* as an element

reached ← a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node ← POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then** **return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s ← *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] ← *child*

 add *child* to *frontier*

return failure

function EXPAND(*problem*, *node*) **yields** nodes

s ← *node*.STATE

for each *action* **in** *problem*.ACTIONS(*s*) **do**

s' ← *problem*.RESULT(*s*, *action*)

cost ← *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

```

Algoritmo A*
def a_star_search(grid, src, dest, numRows, numCols):
    # Inicializa cada célula
    cell_details = [[Cell() for _ in range(numCols)] for _ in range(numRows)]
    # Inicializa os parâmetros da célula de origem
    i = src[0]
    j = src[1]
    cell_details[i][j].f = 0
    cell_details[i][j].g = 0
    cell_details[i][j].h = 0
    cell_details[i][j].parent_i = i
    cell_details[i][j].parent_j = j
    # Inicializa a lista aberta (células a serem visitadas) com a célula de origem
    open_list = []
    heapq.heappush(open_list, (0.0, i, j))
    # Inicializa a flag para saber se o destino foi encontrado
    found_dest = False
    # Inicializa a lista fechada (células já visitadas)
    closed_list = [[False for _ in range(numCols)] for _ in range(numRows)]
    # Main loop of A* search algorithm
    while len(open_list) > 0:
        # Obtém a célula com o menor valor f da lista aberta
        p = heapq.heappop(open_list)
        # Marca a célula como visitada
        i = p[1]
        j = p[2]
        closed_list[i][j] = True
        if is_destination(i, j, dest):
            print("A célula de destino foi encontrada")
            # Traça e imprime o caminho da origem até o destino
            trace_path(cell_details, dest)
            found_dest = True
            return
        # For each direction, check the successors
        # Para cada direção (incluindo as diagonais), verifique os sucessores
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0), (1, 1), (1, -1), (-1, 1), (-1, -1)]
        for dir in directions:
            new_i = i + dir[0]
            new_j = j + dir[1]
            # Se o sucessor é válido, está livre e não foi visitado
            if is_valid(new_i, new_j, numRows, numCols) and is_unblocked(grid, new_i, new_j) and not closed_list[new_i][new_j]:
                # Calcula os novos valores de f, g e h
                g_new = cell_details[i][j].g + 1.0
                h_new = calculate_h_value(new_i, new_j, dest)
                f_new = g_new + h_new
                # Se a célula não está na lista aberta ou o novo valor de f é menor
                if cell_details[new_i][new_j].f == float('inf') or cell_details[new_i][new_j].f > f_new:
                    # Adicione a célula para a lista aberta
                    heapq.heappush(open_list, (f_new, new_i, new_j))
                    # Atualiza os parâmetros da célula
                    cell_details[new_i][new_j].f = f_new
                    cell_details[new_i][new_j].g = g_new
                    cell_details[new_i][new_j].h = h_new
                    cell_details[new_i][new_j].parent_i = i
                    cell_details[new_i][new_j].parent_j = j
    # Se o objetivo não foi encontrado depois de visitar todas as células
    if not found_dest:
        print("Failed to find the destination cell")

```

